



河南大學
HENAN UNIVERSITY

计算机操作系统

主讲人：胡 萍

明德，新民

止于至善



第二章 进程的描述与控制

第4讲





知识回顾

□ 临界资源、临界区、进入区、退出区

```
While(TRUE)
```

```
{
```

```
    进入区 (entry section)    //申请资源，成功则加锁
```

```
    临界区 (critical section) //访问临界资源
```

```
    退出区 (exit section)    //释放资源，解锁
```

```
    :
```

```
}
```



知识回顾

- 进程同步机制应遵循的4条准则
- 信号量机制：一种卓有成效的**进程同步机制**，实现**临界资源**的**互斥共享**和**进程相互合作（同步）**，从而程序执行具有**可再现性**。
- 信号量机制的发展历程：
整形信号量、记录型信号量、AND型信号量、信号量集



知识回顾

□ 整型信号量：表示可用资源数目的整型变量S

- S 除初始化外，仅能通过wait(S)和signal(S) 访问；
- wait(S)又称P操作，用于申请资源（不成功则一直申请），相当于进入区；
- signal(S)又称V操作，用于释放资源，相当于退出区；
- 缺点：未遵循“让权等待”，会发生“忙等”



知识回顾

- 记录型信号量：采用记录型数据结构 **S**，包括一个表示可用资源数目的整型变量 **value**，以及一个阻塞队列链表指针 **list**。
 - **S** 除初始化外，仅能通过 **wait(S)** 和 **signal(S)** 访问；
 - **wait(S)** 又称 **P** 操作，用于申请资源（不成功则阻塞自己），相当于进入区；
 - **signal(S)** 又称 **V** 操作，用于释放资源（同时唤醒阻塞队列中的第一个进程），相当于退出区；
 - 问题：每次仅申请或释放一种资源



知识回顾

□ **AND型信号量**：记录型信号量的扩展，多个资源的申请或释放采用原子分配，即一次分配所有资源，缺一就不分配。

- **Swait($S_1, \dots, S_n$)**和**Ssignal($S_1, \dots, S_n$)**
- 问题：每次仅申请或释放时，申请个数和释放个数为1

□ **信号量集**：**AND型信号量**的扩展。

- 资源分配的下限值 t_i ，则 $S_i \geq t_i$ 时，允许分配资源；
- 资源需求量 d_i ，则申请资源时，进行 $S_i = S_i - d_i$ ；
- **Swait($S_1, t_i, d_i, \dots, S_n, t_n, d_n$)**和 **Ssignal($S_1, t_i, d_i, \dots, S_n, t_n, d_n$)**



知识回顾

□ 利用信号量实现进程互斥

互斥问题

- (1) 分析问题，确定进程数、进程活动以及临界区；
- (2) 设置互斥信号量mutex，初值为1；
- (3) 临界区前对信号量进行wait操作（P操作）；
- (4) 临界区后对信号量进行signal操作（V操作）。

```
semaphore mutex=1;
```

```
Pi( ){  
    while(true){  
        wait(mutex);  
        临界区;  
        signal(mutex);  
        ...  
    }  
}
```


16. 用P、V操作管理临界区时，信号量的初值应定义为（ ）

- ☐ A -1
- ☐ B 0
- ☒ C 1
- ☐ D 任意值

提交



17. 有 m 个进程共享同一临界资源，若使用信号量机制实现对临界资源的互斥访问，则信号量值的变化范围是
()

- ☒ A $[-(m-1), 1]$
- ☐ B $[-m, 1]$
- ☐ C $[-(m-1), 1]$
- ☐ D $[-m, 1)$

提交





1. 利用信号量实现进程互斥

【举例】某交通路口设置一个自动计数系统，系统由“观察者”进程和“报告者”进程组成。观察者进程能识别卡车，并对通过的卡车计数；报告者进程定时将观察者的计数值打印输出，每次打印后把计数值清“0”。两个进程的并发执行可完成对每小时中卡车流量的统计。

```
Observer{
```

```
    计数值加1;
```

```
}
```

```
Reporter{
```

```
    打印计数值;
```

```
    计数值清零;
```

```
}
```

(1) 确定进程数、进程活动以及临界资源；计数值count

确定临界区 访问count的语句，包括累加、打印和清零



1. 利用信号量实现进程互斥

```
struct semaphore mutex ;  
int count = 0 ;  
mutex.value = 1 ;
```

临界区

```
Observer{  
    while (true){  
        observe a lorry;  
        wait(mutex);  
        count = count + 1;  
        signal(mutex);  
    }  
}
```

```
Reporter{  
    while (true){  
        wait(mutex);  
        print (count) ;  
        count = 0;  
        signal(mutex);  
    }  
}
```

临界区



2. 利用信号量实现前趋关系

同步问题

□ 假设有并发执行的进程P1和P2。若希望先执行A，再执行B。

为实现这种前驱关系，需要进行如下操作： $A \xrightarrow{S1} B$

- 进程P1和P2共享一个同步信号量S1，初值为0；
- 语句A后进行signal(S1)操作，语句B前进行wait(S1)操作。

```
P1( ){  
    A ;  
    signal(S1);  
}
```

```
P2( ){  
    wait(S1);  
    B ;  
}
```

S1初值为0，若P2先执行必定阻塞

进程P1执行signal(S1)操作后，S1增为1，P2方能执行语句B。



2. 利用信号量实现前趋关系

同步问题

□ 利用信号量实现前趋关系（进程同步）方法总结：

- （1）分析问题，确定进程数、进程活动，找到需要实现“一前一后”前趋关系（进程同步）的位置；
- （2）在**每个位置**上设置同步信号量，**初始值为0**；
- （3）在“前操作”之后执行**signal操作**（V操作）；
- （4）在“后操作”之前执行**wait操作**（P操作）。



2. 利用信号量实现前趋关系

同步问题

□ 一次同步执行：

执行顺序：A $\xrightarrow{S1}$ B

单向
一次性

```
struct semaphore S1;  
S1.value=0;
```

```
P1( ){  
    A ;  
    Signal(S1);  
}
```

先运行的进程

```
P2( ){  
    Wait(S1);  
    B ;  
}
```

后运行的进程

思考：为什么是一次性的？如何实现循环同步？

A $\rightleftarrows$ B



2+. 利用信号量实现循环同步

□ 循环同步执行： 执行顺序： $A \xrightleftharpoons[S2]{S1} B$

按照一定顺序
循环运行

```
struct semaphore S , S2;  
S1.value=0;  
S2.value=1;
```

```
P1( ){  
    wait(S2) ;  
    A;  
    signal(S1);  
}
```

```
P2( ){  
    wait(S1);  
    B;  
    signal(S2) ;  
}
```



2. 利用信号量实现前趋关系

同步问题

- 思考：假设有并发执行的进程P1和P2，如图所示。要求按照下列顺序：A B D E F C

S1、S2的初始值为0

P1() {

A;

B;

signal(S1);

wait(S2);

C;

}

P2() {

wait(S1);

D;

E;

F;

signal(S2);

}

程序内部顺序性

程序外部异步性
信号量S1进行同步

程序内部顺序性

程序外部异步性
信号量S2进行同步

程序内部顺序性

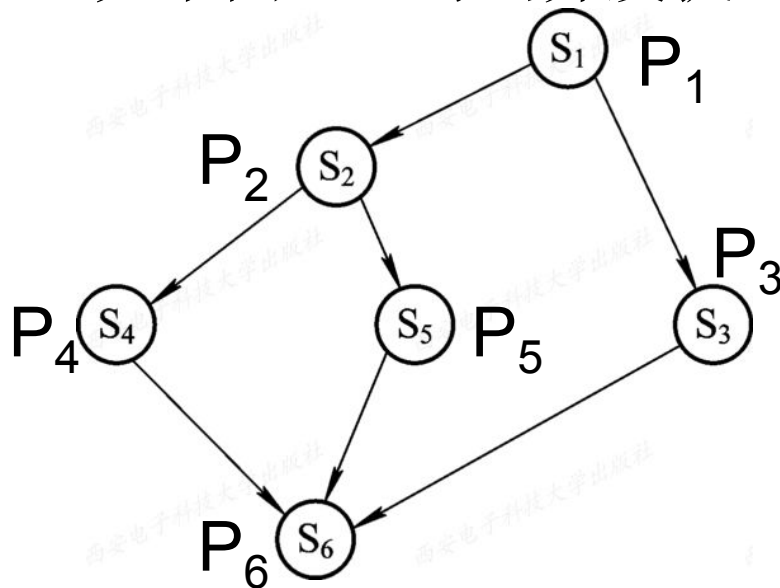




2. 利用信号量实现前趋关系

□ 利用信号量按照语句间的前趋关系（见下图），写出并发执行的程序。

- 进程P1中有语句S1;
- 进程P2中有语句S2;
-
- 语句S1执行后才能执行语句S2和语句S3;
- 语句S2执行后才能执行语句S4和S5;
- 语句S3、S4和S5执行后，才能执行语句S6。



P1执行完应通知P2、P3;
P2得到通知后才开始执行;
P2执行完应通知P4、P5;
.....



2. 利用信号量实现前趋关系

□ 对应于每一对前趋关系，应分别设置信号量。

$S_1 \rightarrow S_2$: a

$S_1 \rightarrow S_3$: b

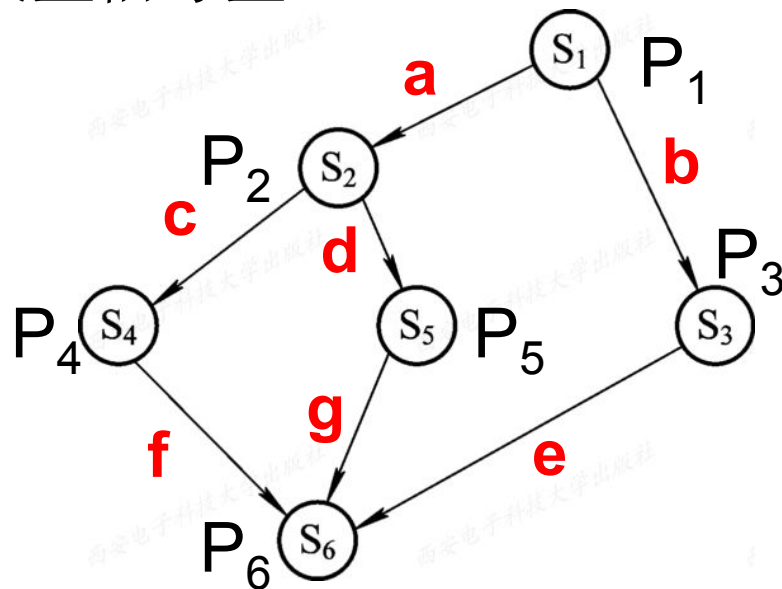
$S_2 \rightarrow S_4$: c

$S_2 \rightarrow S_5$: d

$S_3 \rightarrow S_6$: e

$S_4 \rightarrow S_6$: f

$S_5 \rightarrow S_6$: g



P1执行完应通知P2、P3;
P2得到通知后才开始执行;
P2执行完应通知P4、P5;
.....



2. 利用信号量实现前趋关系

```
struct semaphore a, b, c, d, e, f, g ;
    a.value = b.value
= c.value = d.value
= e.value = f.value
= g.value = 0;
```

初始化

begin /*begin表示并发执行开始*/

P1: { S1; signal(a); signal(b); }

P2: { wait(a); S2; signal(c); signal(d); }

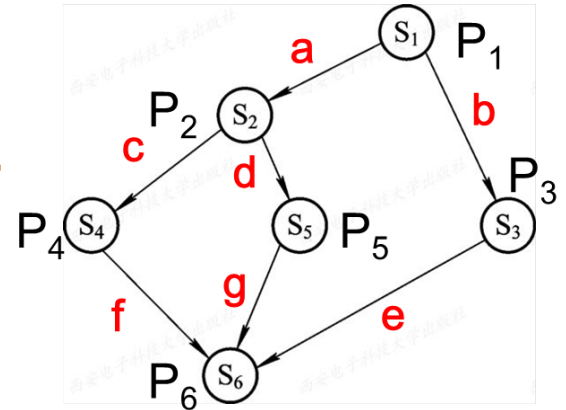
P3: { wait(b); S3; signal(e); }

P4: { wait(c); S4; signal(f); }

P5: { wait(d); S5; signal(g); }

P6: { wait(e); wait(f); wait(g); S6; }

end /*用end表示并发执行结束 */



$S_1 \rightarrow S_2: a$
 $S_1 \rightarrow S_3: b$
 $S_2 \rightarrow S_4: c$
 $S_2 \rightarrow S_5: d$
 $S_3 \rightarrow S_6: e$
 $S_4 \rightarrow S_6: f$
 $S_5 \rightarrow S_6: g$



3. 利用信号量控制使用资源进程的数量

- 系统中有5台打印机可以使用，请使用进程同步机制使得最多可以有5个进程可以同时使用打印机，多于5个进程使用时，要对新申请进程进行阻塞

程序描述如下：

```
struct semaphore S;  
S.value=5 ;  
Printer( ){  
    wait(S);  
    print the document on the paper;  
    signal(S);  
}
```


2.4.4 信号量的应用

习题课





信号量应用解题思路

- Step 1: 分析问题，确定进程数以及各进程的活动；
- Step 2: 确定进程间的关系：互斥、同步、资源管控问题；
- Step 3: 确定信号量个数及初值；
- Step 4: 将wait、signal操作放到合适的位置。



习题1

```
P1( ){  
    拣白子;  
}
```

S1

S2

```
P2( ){  
    拣黑子;  
}
```

在一个盒子里，混装了数量相等的黑白围棋子。现在用自动分拣系统把黑子、白子分开，设分拣系统有两个进程P1和P2，其中P1拣白子，P2拣黑子。规定当一个进程拣了一子后，必须让另一个进程去拣。用信号量和wait、signal操作协调两进程的活动。

灵魂3问：

● 互斥问题or同步问题or资源管控问题？

同步

● 需要几个信号量？

2个，需要循环执行

● 初值如何设置？

分别设置为0和1



习题1解答



```
struct semaphore S1,  
S2;  
S1.value=0;  
S2.value=1;
```

wait(S1)即P(S1),
signal(S1) 即V(S1)

```
P1( ){  
    while(true){  
        wait(S2);  
        拣白子( );  
        signal(S1);  
    }  
}
```

```
P2( ){  
    while(true){  
        wait(S1);  
        拣黑子( );  
        signal(S2);  
    }  
}
```



习题2

1个生产者，1个消费者，1个存储缓冲区

某控制系统中，数据采集进程负责把采集到的数据放到一个缓冲区中；分析进程负责把数据从缓冲区中取出进行分析，试用信号量实现两者之间的同步。

```
数据采集( ) {  
    采集数据;  
    放入缓冲区;  
}
```

```
数据分析( ) {  
    从缓冲区取数据;  
    分析数据;  
}
```

full

empty

灵魂3问：

● 互斥问题or同步问题or资源管控问题？

同步

● 需要几个信号量？

2个，需要循环执行

● 初值如何设置？

分别设置为0和1



习题2解答



```
struct semaphore full,  
empty;  
full.value = 0;  
empty.value = 1;
```

```
P1( ){  
    while(true){  
        采集数据;  
        wait(empty);  
        放入缓冲区;  
        signal(full);  
    }  
}
```

```
P2( ){  
    while(true){  
        wait(full);  
        从缓冲区中取数据;  
        signal(empty);  
        分析数据;  
    }  
}
```

```
readeri( ){  
    登记;  
    读书;  
    注销;  
}
```



习题3

图书馆规定，每位进入图书馆的读者要在登记表上登记，退出时要在登记表上注销。

- (1) 用信号量实现读者之间的互斥登记和注销;
- (2) 图书馆共有100个座位，当图书馆中没有空座位时，后到的读者在图书馆要等待（阻塞）。

灵魂3问：

互斥和资源管控

- 互斥问题or同步问题or资源管控问题？
- 需要几个信号量？
- 初值如何设置？

2个，1个互斥，1个资源管控

分别设置为1和100



习题3解答

```
struct semaphore S, mutex;  
S.value=100;      /*100个座位资源*/  
mutex.value=1;    /*互斥信号量*/  
  
-----  
readeri( ){  
    while(true){  
        wait (S);      /*先申请资源信号量*/  
        wait (mutex); /*后申请互斥信号量，即登记表的使用权*/  
        登记;  
        signal (mutex);  
        读书;  
        wait (mutex);  
        注销;  
        signal (mutex);  
        signal (S);    /*释放资源信号量*/  
    }  
}
```



习题4

多个生产者，多个消费者，1个存储缓冲区

一家四人父、母、儿子、女儿围桌而坐，桌上有一个水果盘，每次只能向其中放入一个水果。

- 当水果盘空时，父亲可以放香蕉，或母亲可以放苹果
- 盘中已有水果时，就不能继续放，父母等待。
- 当盘中有香蕉时，女儿吃香蕉，否则，女儿等待；
- 当盘中有苹果时，儿子吃苹果，否则，儿子等待。

```
Father( ){  
    准备香蕉;  
    放香蕉;  
}
```

```
Daughter( ){  
    从盘中取香蕉;  
    吃香蕉;  
}
```

```
Mom( ){  
    准备苹果;  
    放苹果;  
}
```

```
Son( ){  
    从盘中取苹果;  
    吃苹果;  
}
```

B



A





习题4解答

- 果盘为空，父母可以操作；果盘非空，父母不能操作。
需要**1个互斥信号量**，表示果盘使用权 **mutex**
- 父亲操作后，若果盘有香蕉，女儿可以从盘中取香蕉。
需要**1个同步信号量** **B**
- 母亲操作后，若果盘有苹果，儿子可以从盘中取苹果。
需要**1个同步信号量** **A**

□ 【小结】

共需要1个互斥信号量和2个同步信号量
互斥信号量mutex初值为1；
同步信号量B，A初值为0。

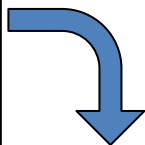


习题4解答

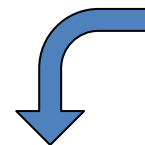
P()即wait() ,
V()即signal()

```
struct semaphore A, B, mutex;  
A.value = B.value = 0;    /*同步信号量*/  
mutex.value=1;           /*互斥信号量*/
```

```
Father( ){  
    while(1){  
        准备香蕉;  
        P(mutex);  
        放香蕉;  
        V(B);  
    }  
}
```



```
Daughter( ){  
    while(1){  
        P(B);  
        从盘中取香蕉;  
        V(mutex);  
        吃香蕉;  
    }  
}
```



```
Son( ){  
    while(1){  
        P(A);  
        从盘中取苹果;  
        V(mutex);  
        吃苹果;  
    }  
}
```

```
Mom( ){  
    while(1){  
        准备苹果;  
        P(mutex);  
        放苹果;  
        V(A);  
    }  
}
```



习题5

在公共汽车上，司机和售票员的活动分别是：

司机的活动： 启动车辆

正常运行

到站停车

售票员的活动： 关车门

售票

开车门

在汽车不断的到站，停车，行驶过程中，司机和售票员的活动有什么同步关系？用信号量和P、V操作实现。

 习题5解答

- 售票员关车门后，司机才能启动车辆。
需要1个同步信号量；**S1**
 - 司机：启动车辆后，正常运行，到站停车。不需要信号量。
 - 司机到站停车后，售票员才能打开车门。
需要1个同步信号量；**S2**
 - 售货员：关车门后就可以开始售票。不需要信号量。
- 【小结】共需要2个同步信号量。初值如何设置？

同步信号量**S1**，**S2**初值为0。



习题5解答



```
struct semaphore S1, S2;  
S1.value = 0;  
S2.value = 0;
```

```
司机( ){  
    while(true){  
        wait(S1);  
        启动车辆;  
        正常运行;  
        到站停车;  
        signal(S2);  
    }  
}
```

```
售票员( ){  
    while(true){  
        关车门;  
        signal(S1);  
        售票;  
        wait(S2);  
        开车门;  
    }  
}
```




习题6

有一个超市，最多可容纳 N 个人进入购物，当 N 个顾客满员时，后到的顾客在超市外等待；超市中只有一个收银员。可以把**顾客和收银员**看作**两类进程**，两类进程间存在**同步**关系。写出用P、V操作实现的两类进程的算法。

顾客活动：

进入超市；
购物；
结账；
离开；

收银员活动：

收银；



习题6解答2

- 超市能容纳N个顾客：需要1个资源信号量S；
 - 收银员同一时间只能为1位顾客结账：需要1个互斥信号量mutex；
 - 顾客购物结束后，找收银员结账：需要1个同步信号量S1；
 - 收银员收银结束后，顾客可以离开：需要1个同步信号量S2；
- 【小结】共需要4个信号量：1个资源信号量S，1个互斥信号量mutex，2个同步信号量S1和S2

```
struct semaphore S, S1, S2, mutex;  
S.value = N;    /*S用于资源管控*/  
S1.value = 0;   /*S1用于顾客结账前和收银员收银的同步*/  
S2.value = 0;   /*S2用于收银员结账后和顾客离开的同步*/  
mutex.value = 1; /*mutex用于收银员互斥*/
```



习题6解答

P()即wait() ,
V()即signal()

```
struct semaphore S, S1, S2, mutex;  
S.value = N;    /*S用于资源管控*/  
S1.value = 0;   /*S1用于顾客结账前和收银员收银的同步*/  
S2.value = 0;   /*S2用于收银员结账后和顾客离开的同步*/  
mutex.value = 1; /*mutex用于收银员互斥*/
```

```
顾客i( ){  
    while(true){  
        P(S); //申请进入  
        进入超市;  
        购物;  
        P(mutex); //申请结账权限  
        V(S1); //通知收银员收银  
        结账;  
        P(S2); //判断是否收银结束  
        V(mutex); //释放结账权限  
        离开;  
        V(S); //离开, 下一位进入  
    }  
}
```

```
收银员{  
    while(true){  
        P(S1); //判断是否可以收银  
        收银;  
        V(S2); //收银结束  
    }  
}
```



2.4.5 管程机制（自学）

- 1. 管程的定义
- 2. 条件变量



WARNING

为了便于阅读和编写程序，从本节课开始将wait(S)替换为P(S)，将signal(S)替换为V(S)。



2.5 经典进程的同步问题

□ 2.5.1 生产者-消费者问题

□ 2.5.2 哲学家进餐问题

□ 2.5.3 读者-写者问题



2.5.1 生产者-消费者问题

- 1. 利用记录型信号量解决生产者-消费者问题
- 2. 利用AND信号量解决生产者-消费者问题

第二章 进程的描述与控制

第5讲





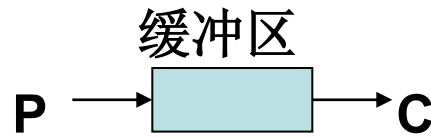
1. 利用记录型信号量解决生产者-消费者问题

- ❑ 前面对生产者-消费者问题的描述，未考虑进程的互斥和同步问题，因而造成了共享变量counter（缓冲池中产品数目）的不确定性。
- ❑ 生产者-消费者问题是相互合作进程关系的一种抽象。

生产者-消费者问题从特殊到一般（从易到难）可分3种形式：

- 一个生产者、一个消费者、一个缓冲区的问题；
- 一个生产者、一个消费者、 n 个缓冲区的问题；
- k 个生产者、 m 个消费者、 n 个缓冲区的问题；

先介绍
最简单的
P-C
问题



一个生产者、一个消费者、一个缓冲区的P-C问题

(1) 确定进程数及进程活动;

(2) 确定进程关系:

循环同步

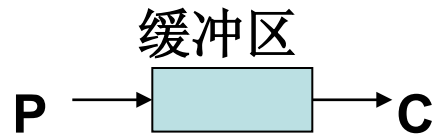
```
Producer( ){  
    生产产品;  
    放入缓冲区;  
}
```

```
Consumer( ){  
    从缓冲区取产品;  
    消费产品;  
}
```

full

empty

- 生产者: 当缓冲区**空**, 将产品存入缓冲区; 当缓冲区满, 必须等待 (**阻塞**), 待**消费者取走产品后将其唤醒**后, 才能将产品存入。
- 消费者: 当缓冲区**满**, 从缓冲区取产品; 当缓冲区空, 必须等待 (**阻塞**), 待**生产者存入产品后将其唤醒**后, 才能从缓冲区取产品。



一个生产者、一个消费者、一个缓冲区的P-C问题

(3) 确定信号量个数及初值;

```
Producer( ){
```

```
    生产产品;
```

```
    放入缓冲区;
```

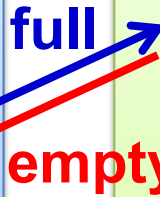
```
}
```

```
Consumer( ){
```

```
    从缓冲区取产品;
```

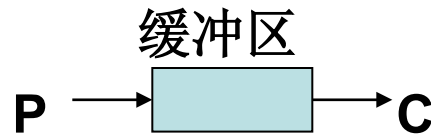
```
    消费产品;
```

```
}
```



循环同步

- 同步信号量**full**: 表示满缓冲区的数量, 初值为0, 表示开始时没有满缓冲区。
- 同步信号量**empty**: 表示空缓冲区的数量, 其初值为1, 表示有1个空缓冲区。



一个生产者、一个消费者、一个缓冲区的P-C问题

(4) 将P、V操作放到合适的位置。

```
Producer( ){  
    生产产品;  
    P(empty);  
    放入缓冲区  
    V(full);  
}
```

```
Consumer( ){  
    P(full);  
    从缓冲区取产品;  
    V(empty);  
    消费产品;  
}
```

循环同步

- 生产者：产品存入缓冲区之前，执行**P(empty)**操作，申请空缓冲区；存入产品后，执行**V(full)**操作，通知（可能会唤醒）消费者。
- 消费者：从缓冲区取产品前，执行**P(full)**操作，申请满缓冲区；取走产品后，执行**V(empty)**操作，通知（可能会唤醒）生产者。

一个生产者、一个消费者、一个缓冲区P-C问题伪代码

```
item buffer; //共享数据
struct semaphore empty, full; //信号量
empty.value=1;
full.value=0;
```

```
Producer( ){
    while(true){
        produce an item in nextp;
        P(empty); //申请空缓冲区
        buffer = nextp;
        V(full); //通知消费者消费
    }
}
```

```
Consumer( ){
    while(true){
        P(full); //申请满缓冲区
        nextc = buffer;
        V(empty); //通知生产者生产
        consume the item in nextc;
    }
}
```



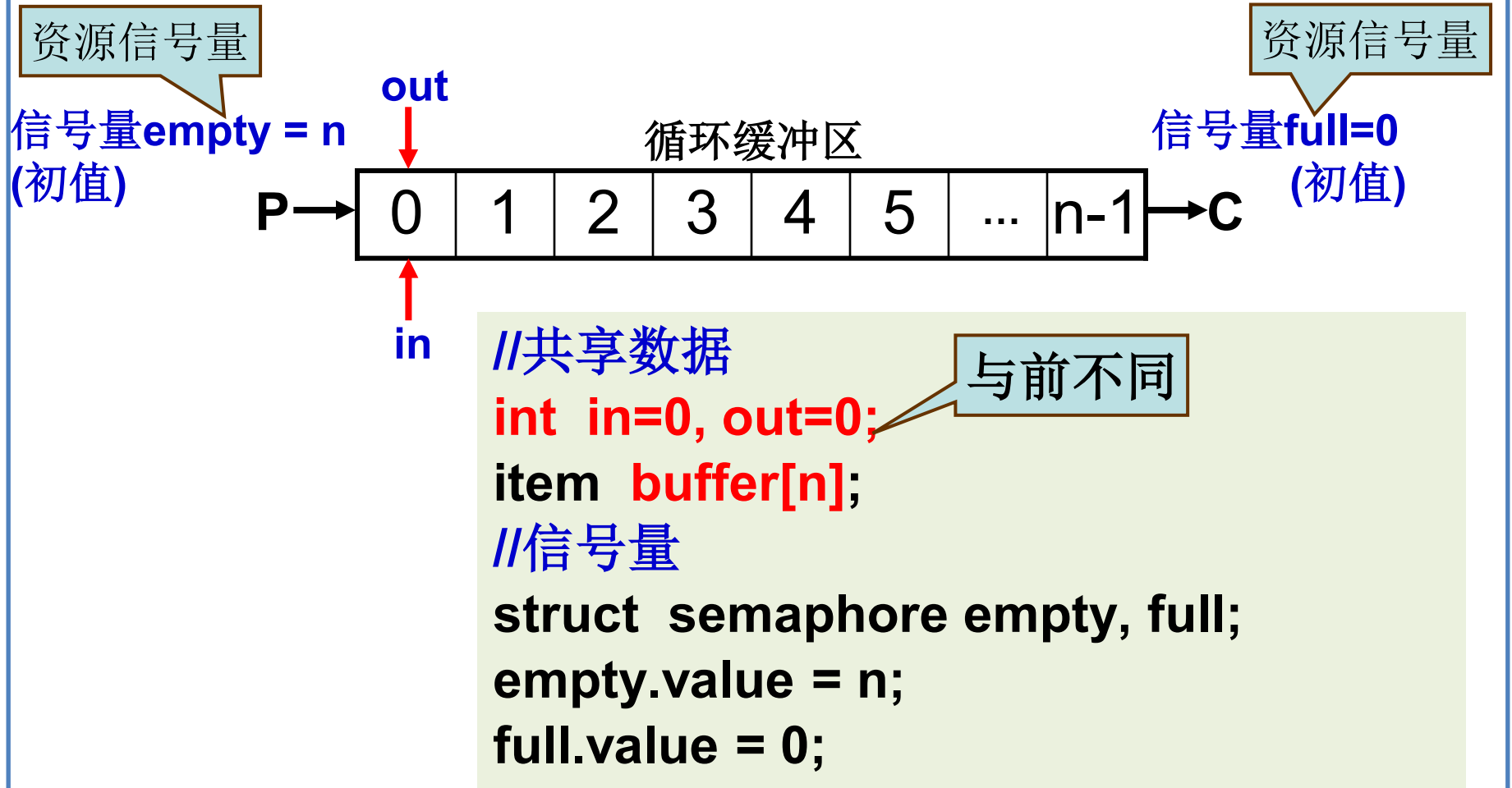
一个生产者、一个消费者、 n 个缓冲区的P-C问题



- (1) 确定进程数及进程活动;
- (2) 确定进程关系: **循环同步关系**
- (3) 确定信号量个数及初值;
- (4) 将P、V操作放到合适的位置。



一个生产者、一个消费者、 n 个缓冲区的P-C问题



一个生产者、一个消费者、n个缓冲区P-C问题伪代码

```
int in=0, out=0; //共享数据
item buffer[n];
struct semaphore empty, full; //信号量
empty.value = n;
full.value = 0;
```

```
Producer( ){
    while(true){
        produce an item in nextp;
        P(empty); //申请空缓冲区
        buffer[in] = nextp;
        in = (in+1) % n;
        V(full); //满缓冲区加1
    } }
```

```
Consumer( ){
    while(true){
        P(full); //先申请满缓冲区
        nextc = buffer[out];
        out = (out+1) % n;
        V(empty); //空的缓冲区加1
        consume the item in nextc;
    } }
```



一个生产者、一个消费者、n个缓冲区P-C问题伪代码，可行？

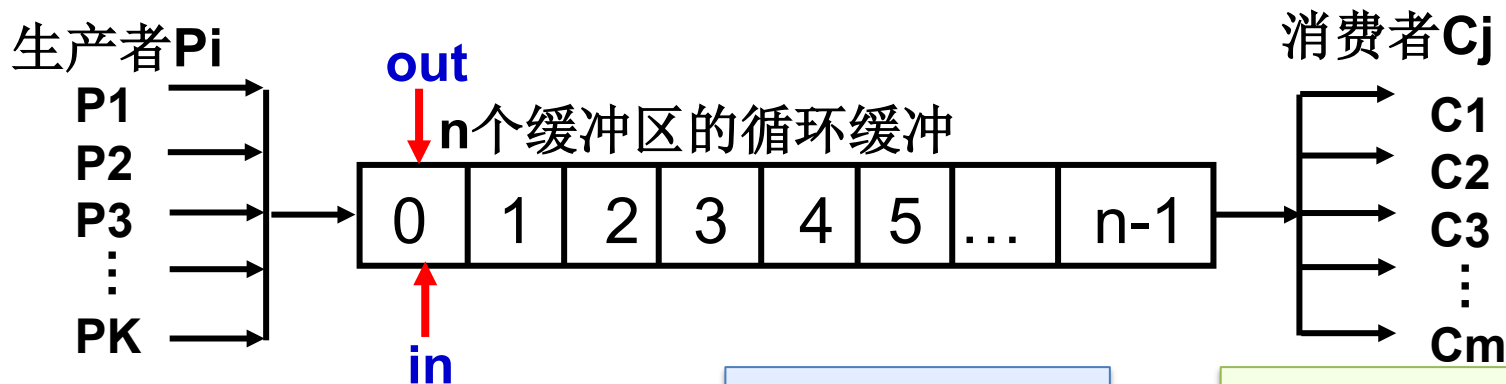
```
int in=0, out=0; //共享数据
item buffer[n];
struct semaphore empty, full, mutex; //信号量
mutex.value = 1;
empty.value = n;
full.value = 0;
```

```
Producer( ){
    while(true){
        produce an item in nextp;
        P(empty); //先申请资源信号量
        P(mutex); //再申请互斥信号量
        buffer[in] = nextp;
        in = (in+1) % n;
        V(mutex);
        V(full); //满缓冲区加1
    } }
```

```
Consumer( ){
    while(true){
        P(full); //先申请资源信号量
        P(mutex); //再申请互斥信号量
        nextc = buffer[out];
        out = (out+1) % n;
        V(mutex);
        V(empty); //空的缓冲区加1
        consume the item in nextc;
    } }
```

同类进程对缓冲池互斥使用。即生产者进程用共享变量in指针访问缓冲区的互斥、消费者进程用共享变量out指针访问缓冲区的互斥。

● k个生产者、m个消费者、n个缓冲区的P-C问题



(1) 确定进程数及进程活动;

```
Produceri( ){  
    生产产品;  
    放入缓冲区;  
}
```

```
Consumerj( ){  
    从缓冲区取产品;  
    消费产品;  
}
```

full (blue arrow pointing right)
empty (red arrow pointing left)

(2) 确定进程关系: 异类进程循环同步关系、同类进程互斥关系

(3) 确定信号量个数及初值; 信号量: **empty**, 初值= n ; **full**, 初值= 0

互斥信号量 **mutexp**, 初值为 1

(4) 将P、V操作放到合适的位置。互斥信号量 **mutexc**, 初值为 1

k个生产者、m个消费者、n个缓冲区P-C问题伪代码

```
int in=0, out=0; //共享数据
item buffer[n];
struct semaphore empty, full, mutexp, mutexc; //信号量
mutexp.value = mutexc.value = 1;
empty.value = n;
full.value = 0;
```

```
Produceri( ){
    while(true){
        produce an item in nextp;
        P(empty); //先申请资源信号量
        P(mutexp); //再申请互斥信号量
        buffer[in] = nextp;
        in = (in+1) % n;
        V(mutexp);
        V(full); //满缓冲区加1
    } }
```

```
Consumerj( ){
    while(true){
        P(full); //先申请资源信号量
        P(mutexc); //再申请互斥信号量
        nextc = buffer[out];
        out = (out+1) % n;
        V(mutexc);
        V(empty); //空的缓冲区加1
        consume the item in nextc;
    } }
```



k个生产者、m个消费者、n个缓冲区P-C问题伪代码可行？

```
int in=0, out=0; //共享数据
item buffer[n];
struct semaphore empty, full, mutex; //信号量
mutex.value = 1;
empty.value = n;
full.value = 0;
```

```
Produceri( ){
    while(true){
        produce an item in nextp;
        P(empty); //先申请资源信号量
        P(mutex); //再申请互斥信号量
        buffer[in] = nextp;
        in = (in+1) % n;
        V(mutex);
        V(full); //满缓冲区加1
    } }
```

```
Consumerj( ){
    while(true){
        P(full); //先申请资源信号量
        P(mutex); //再申请互斥信号量
        nextc = buffer[out];
        out = (out+1) % n;
        V(mutex);
        V(empty); //空的缓冲区加1
        consume the item in nextc;
    } }
```



生产者-消费者问题

- ❑ 每个进程中，实现**互斥的P(mutex)和V(mutex)**必须成对出现；
- ❑ 对**资源信号量empty和full的P和V操作**也要成对地出现，但它们处于**不同的进程**中。
- ❑ 在每个进程中的多个**P操作**顺序不能颠倒，应**先执行对资源信号量的P操作**，然后**执行对互斥信号量的P操作**，否则可能引起进程**死锁**。



2. 利用AND信号量解决生产者-消费者问题

□ k个生产者、m个消费者、n个缓冲区生产者-消费者问题

```
int in=0, out=0; //共享数据
item buffer[n];
struct semaphore empty, full, mutex; //信号量
mutex.value = 1;
empty.value = n;
full.value = 0;
```




2. 利用AND信号量解决生产者-消费者问题

```
Produceri( ){  
    while(true){  
        produce an item in nextp;  
        Swait(empty, mutex)  
        buffer[in] = nextp;  
        in = (in+1) % n;  
        Ssignal(mutex, full)  
    }  
}
```

```
Consumerj( ){  
    while(true){  
        Swait(full, mutex)  
        nextc = buffer[out];  
        out = (out+1) % n;  
        Ssignal(mutex, empty)  
        consume the item in nextc;  
    }  
}
```



2.5.3 读者-写者问题

- 1. 利用记录型信号量解决读者-写者问题
- 2. 利用信号量集解决读者-写者问题（自学）



2.5.3 读者-写者问题

□ 动机：共享数据的访问

典型例子：航班预订系统的数据库

□ 问题描述：

- 读者进程：不需要修改数据；
- 写者进程：读取和修改数据；
- 不允许两个或两个以上写者进程同时访问共享文件；
- 不允许写者进程和读者进程同时访问共享文件；
- 允许多个读者进程同时读一个共享文件；

所谓“读者-写者问题”，是指保证一个写（**writer**）进程必须与其它进程互斥地访问共享对象的问题。



1. 利用记录型信号量解决读者-写者问题

□ 读者-写者问题从易到难分3种形式：



- 一个读者、一个写者
- k 个读者、一个写者
- k 个读者、 m 个写者



一个读者、一个写者的读者-写者问题

- (1) 确定进程数及进程活动;
- (2) 确定进程间的关系; **读写进程互斥访问文件**
- (3) 确定信号量个数及初值; **wmutex, 初值=1**
- (4) 将P、V操作放到合适的位置。

```
Reader( ){  
    while(true){  
        P(wmutex);  
        reading;  
        V(wmutex);  
    }  
}
```

```
Writer( ){  
    while(true){  
        P(wmutex);  
        writing;  
        V(wmutex);  
    }  
}
```



一读者、一写者的读者-写者问题伪代码

semaphore wmutex;//互斥信号量，实现读写进程对文件互斥访问
wmutex.value = 1;

```
Reader( ){  
    P(wmutex);//申请文件使用权  
    reading;  
    V(wmutex);  
}
```

```
Writer( ){  
    P(wmutex); //申请文件使用权  
    writing;  
    V(wmutex);  
}
```



● k个读者、一个写者的读者-写者问题

- (1) 确定进程数及进程活动;
- (2) 确定进程间的关系; **读写互斥、读读可同时共享文件**
- (3) 确定信号量个数及初值; **wmutex, 初值=1**
- (4) 将P、V操作放到合适的位置。

```
Readeri( ){  
    while(true){  
        P(wmutex);  
        reading;  
        V(wmutex);  
    }  
}
```

```
Writer( ){  
    while(true){  
        P(wmutex);  
        writing;  
        V(wmutex);  
    }  
}
```




● k个读者、一个写者的读者-写者问题

(3) 确定信号量个数及初值;

- 写者进程: 不允许和读者或其他写者进程同时访问共享文件;
互斥关系 互斥信号量wmutex, 初值为1
- 读者进程 { 不允许和写者进程同时访问共享文件;
互斥关系
允许多个读者进程同时访问共享文件。

思考: 第几个读者进程申请使用权? 哪个读者进程释放资源?

共享变量readcount, 表示正在进行的读者进程数, 初值为0

需读者进程互斥访问, 临界资源。

互斥信号量rmutex, 控制readcount使用权, 初值为1



● k个读者、一个写者的读者-写者问题分析

- 互斥信号量**wmutex**：实现读者进程和写者进程、写者进程之间的互斥，其初值为1；
- 仅**第一个读者进程**需要执行P(wmutex)操作，申请文件使用权；
- 仅**最后一个读者进程**离开时，需执行V(wmutex)操作，释放资源；
- 设**整型变量readcount**，记录正在执行的读者进程数，初值为0；
- readcount是被多个读者进程访问的**临界资源**，互斥访问，应设置一个互斥信号量**rmutex**。
- 当readcount = 0时，**下一个要执行的读者进程**需要进行P(wmutex)操作，申请文件使用权；若成功，读者进程可读文件，同时readcount+1。
- 当readcount-1后变为0，当前进程即为最后一个读者进程。



k个读者、一个写者的读者-写者问题

$i=1, \dots, k$

```
int readcount = 0; //共享变量
semaphore wmutex, rmutex; //信号量
wmutex.value=rmutex.value=1;
```

```
Readeri( ){
    while(true){
        P(rmutex); //申请计数值使用权
        if (readcount == 0)
            P(wmutex); //第1个读进程申请文件
        readcount++;
        V(rmutex);
        reading;
        P(rmutex);
        readcount--;
        if (readcount == 0)
            V(wmutex); //最后读进程释放文件
        V(rmutex);
    }
}
```

```
Writer( ){
    while(true){
        P(wmutex); //申请文件使用权
        writing;
        V(wmutex);
    }
}
```



● k个读者、m个写者的读者-写者问题

- (1) 确定进程数及进程活动;
- (2) 确定进程间的关系; **读写互斥、读读可同时共享文件**
- (3) 确定信号量个数及初值; **wmutex, 初值=1**
- (4) 将P、V操作放到合适的位置。

```
Readeri( ){  
    while(true){  
        P(wmutex);  
        reading;  
        V(wmutex);  
    }  
}
```

```
Writerj( ){  
    while(true){  
        P(wmutex);  
        writing;  
        V(wmutex);  
    }  
}
```

▲第一个读者读文件时，后续读者也可读该文件，后续写者不能写(在 **wmutex** 上阻塞)，由最后退出的读者唤醒一个写者。



k个读者、m个写者的读者-写者问题

$i=1, \dots, k; j=1, \dots, m$

读者优先

```
int readcount = 0;
semaphore wmutex, rmutex;
wmutex.value=rmutex.value=1;
```

```
Readeri( ){
    while(true){
        P(rmutex); //申请计数值使用权
        if (readcount == 0)
            P(wmutex); //第1个读进程申请文件
        readcount++;
        V(rmutex);
        reading;
        P(rmutex);
        readcount--;
        if (readcount == 0)
            V(wmutex); //最后读进程释放文件
        V(rmutex);
    }
}
```

```
Writerj( ){
    while(true){
        P(wmutex); //申请文件使用权
        writing;
        V(wmutex);
    }
}
```

▲有一个写者在写时，后续写者不能写，在 **wmutex** 上阻塞；后续读者不能读，其中第一个读者在 **wmutex** 上阻塞，其余读者在 **rmutex** 上阻塞。该写者退出时，唤醒一个写者或读者。

思考：如何实现公平的先来先服务的原则？



k个读者、m个写者的读者-写者问题

$i=1, \dots, k; j=1, \dots, m$

```
int readcount = 0;
semaphore wmutex, rmutex;
semaphore S; //实现读写公平竞争
wmutex.value=rmutex.value= S.value = 1;
```

```
Readeri( ){
    while(true){
        P(S);
        P(rmutex); //申请计数值使用权
        if (readcount == 0)
            P(wmutex); //第1个读进程申请文件
        readcount++;
        V(rmutex);
        V(S);
        reading;
        P(rmutex);
        readcount--;
        if (readcount == 0)
            V(wmutex); //最后读进程释放文件
        V(rmutex);
    }
}
```

```
Writerj( ){
    while(true){
        P(S);
        P(wmutex); //申请文件使用权
        writing;
        V(wmutex);
        V(S);
    }
}
```



2.5.2 哲学家进餐问题

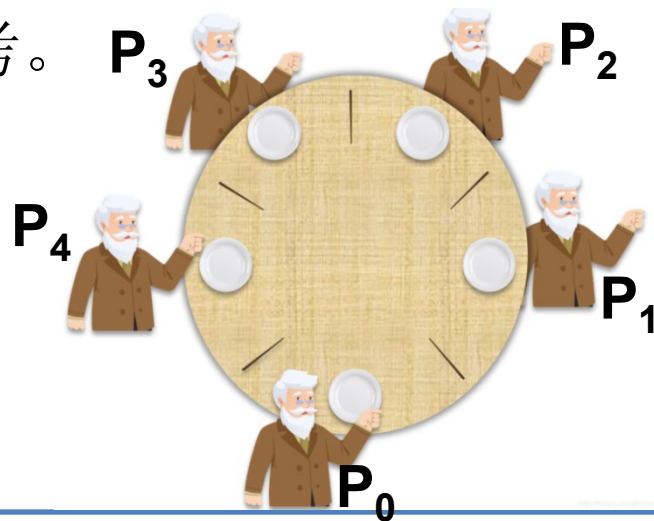
- 1. 利用记录型信号量解决哲学家进餐问题
- 2. 利用AND信号量解决哲学家进餐问题



2.5.2 哲学家进餐问题

□ 哲学家就餐问题描述如下：

5个哲学家共用一张圆桌，分别坐在5张椅子上，在圆桌上有**5个碗**和**5只筷子**。他们的生活方式是交替地进行思考和进餐。平时，每个哲学家进行思考，饥饿时便试图拿起其左右最靠近他的筷子（一根一根的拿），只有在他拿到两只筷子时才能进餐。进餐完毕，放下筷子继续思考。





2.5.2 哲学家进餐问题

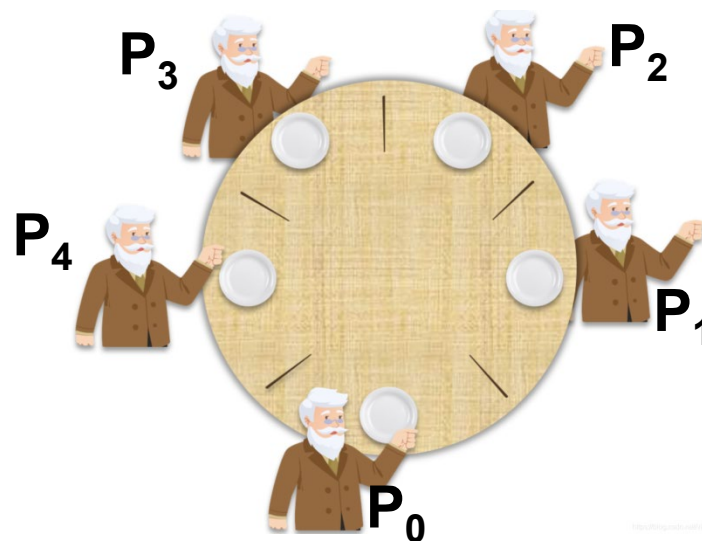
□ 分析：

(1) 确定进程数及进程活动；

5个哲学家进程 P_i ($i=0, \dots, 4$)

哲学家 P_i ：

- (1) 拿起左、右筷子；
- (2) 吃饭；
- (3) 放下左、右筷子；
- (4) 思考问题；
- (5) 返回(1)。

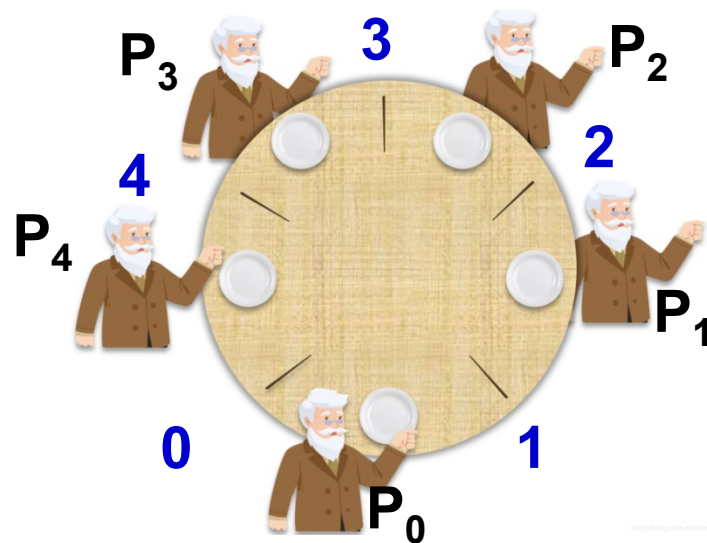


如何避免临界资源分配不当造成的死锁，是哲学家问题的关键。

2.5.2 哲学家进餐问题

(2) 确定进程关系：**互斥关系**

5个哲学家进程 P_i ($i=0, \dots, 4$) 之间对临界资源筷子要**互斥共享**，且每个哲学家进程需要同时持有两个临界资源才能吃饭。





2.5.2 哲学家进餐问题

(3) 确定信号量个数及初值;

桌子上的筷子是临界资源，应互斥使用，可用一个信号量表示一只筷子，5只筷子的5个信号量构成信号量数组，所有信号量的初值均为1。

```
struct semaphore chopstick[5] ;  
chopstick[0].value=chopstick[1].value=1;  
chopstick[2].value=chopstick[3].value=1;  
chopstick[4].value=1 ;
```

初始化

如果5个哲学家并发地拿起了自己左手边的筷子，每位哲学家循环等待右边的人放下筷子（阻塞）。发生“死锁”

1. 利用记录型信号量解决哲学家进餐问题

(4) 将P、V操作放到合适位置：第 i 个($i = 0, \dots, 4$) 哲学家进程：

```
semaphore chopstick[5] = { 1, 1, 1, 1, 1 }
```

```
parbegin
```

```
process Pi ( ) {
```

```
    while (true) {
```

```
        wait(chopstick[i]);
```

//拿起左边筷子

```
        wait(chopstick[(i + 1) % 5]);
```

//拿起右边筷子

```
        eating;
```

```
        signal(chopstick[i]);
```

//放下左边筷子

```
        signal(chopstick[(i+1) % 5]);
```

//放下右边筷子

```
        thinking;
```

```
    }
```

```
}
```

```
parend
```

此算法虽然能保证相邻哲学家对筷子的访问互斥，但可能引起死锁。
(Why?)



● 哲学家进餐死锁问题解决方法

□ 对上述哲学家算法的死锁问题，可采取下面几种解决方法：

- I. 至多允许4个哲学家同时取左边的筷子，这样能至少保证一个哲学家能就餐，并在用毕后释放他用过的两只筷子，从而使更多的哲学家能够进餐。（先自行思考）
- II. 仅当哲学家左右两只筷子均可用时，才允许他拿起筷子进餐。用AND信号量机制
- III. 规定奇数号哲学家先拿左边筷子，然后再拿右边筷子；偶数号哲学家先拿右边筷子，然后再拿左边筷子。



哲学家进餐死锁问题解决方法 I

I. 最多允许 $n = 4$
个哲学家同时
进餐:

```
semaphore n = 4;  
semaphore chopstick [5] = {1, 1, 1, 1, 1};  
Parbegin  
process Pi ( ) {  
    while (true) {  
        wait(n); //控制最多4人同时进餐  
        wait(chopstick[i]);  
        wait(chopstick[(i+1) % 5]);  
        eating;  
        signal(chopstick[i]);  
        signal(chopstick[(i+1) % 5]);  
        signal(n); //用餐完毕释放筷子  
        thinking;  
    }  
}  
parend
```



哲学家进餐死锁问题解决方法III

- III. 奇数号哲学家
先拿其左边的
筷子，然后再
拿其右边的筷
子；偶数号哲
学家先拿其右
边的筷子，然
后再拿其左边
的筷子。

```
semaphore chopstick [5] = {1, 1, 1, 1, 1};
Parbegin
process Pi ( ) {
    while (true) {
        if (i % 2 != 0) {
            wait(chopstick[i]);
            wait(chopstick[(i+1) % 5]);
        }
        else {
            wait(chopstick[(i+1) % 5]);
            wait(chopstick[i]);
        }
        eating;
        signal(chopstick[i]);
        signal(chopstick[(i+1) % 5]);
        thinking;
    }
}parend
```



2. 利用AND信号量解决哲学家进餐问题

□ 第 i ($i = 0, 1, 2, 3, 4$)个哲学家的活动可描述如下:

```
semaphore chopstick[5] = { 1, 1, 1, 1, 1 }
```

```
parbegin
```

```
process Pi ( ) {
```

```
    while (true) {
```

```
        swait(chopstick[i], chopstick[(i + 1) % 5]); //拿起左右筷子
```

```
        eating;
```

```
        ssignal(chopstick[i], chopstick[(i + 1) % 5]); //放下左右筷子
```

```
        thinking;
```

```
    }
```

```
}
```

```
parend
```




习题

有一座东西方向的独木桥；用**P**、**V**操作分别实现以下要求：

- (1) 每次只允许一个人过桥；
- (2) 当独木桥上有自东向西的行人时，同方向的行人可以同时过桥，从西向东的方向，只允许一个人单独过桥。
- (3) 当独木桥上有行人时，同方向的行人可以同时过桥，相反方向的人必须等待。



(1) 每次只允许一个人过桥; 互斥问题

```
semaphore mutex;  
mutex.value=1;  
  
process Pi( ){  
    while(true){  
        P(mutex);  
        过桥;  
        V(mutex);  
    }  
}
```



(2) 当独木桥上有自东向西的行人时，同方向的行人可以同时过桥，从西向东的方向，只允许一个人单独过桥

*/*mutex用于东西方互斥，DXmutex用于自东向西方向计数互斥*/*

semaphore mutex = 1, DXmutex = 1;

int DXcount = 0; *// DXcount用于东向西方向计数*

```
process P东向西( ){  
    while(true){  
        P(DXmutex);  
        if(DXcount == 0 ) P(mutex);  
        DXcount ++;  
        V(DXmutex);  
        过桥;  
        P(DXmutex);  
        DXcount --;  
        if(DXcount == 0 ) V(mutex);  
        V(DXmutex);  
    }  
}
```

```
process P西向东( ){  
    while(true){  
        P(mutex);  
        过桥;  
        V(mutex);  
    }  
}
```



(3) 当独木桥上有行人时，同方向的行人可以同时过桥，相反方向的人必须等待

*/*mutex用于东西方互斥，DXmutex用于东向西方向计数互斥，XDmutex用于西向东方向计数互斥*/*

semaphore mutex = 1, DXmutex = 1, XDmutex = 1;

int DXcount = 0, XDcount = 0;; *// 用于计数*

```
process P东向西( ){  
    while(true){  
        P(DXmutex);  
        if(DXcount == 0 ) P(mutex);  
        DXcount ++;  
        V(DXmutex);  
        过桥;  
        P(DXmutex);  
        DXcount --;  
        if(DXcount == 0 ) V(mutex);  
        V(DXmutex);  
    }  
}
```

```
process P西向东 ( ){  
    while(true){  
        P(XDmutex);  
        if(XDcount == 0 ) P(mutex);  
        XDcount ++;  
        V(XDmutex);  
        过桥;  
        P(XDmutex);  
        XDcount --;  
        if(XDcount == 0 ) V(mutex);  
        V(XDmutex);  
    }  
}
```



2.6 进程通信

- 2.6.1 进程通信的类型
- 2.6.2 消息传递通信的实现方式
- 2.6.3 直接消息传递系统实例



2.6 进程通信

- 进程通信：进程之间的信息交换。
- 进程之间的互斥和同步，交换的信息量少——低级通信。
- 信号量机制作为通信工具不够理想，表现在：
 - 效率低；
 - 通信对用户不透明。
- 进程高级通信：指用户可直接利用**OS**所提供的一组通信命令，高效地传送大量数据的一种通信方式。
- 高级通信过程对用户是透明的，大大减少了通信程序编制的复杂性。

进程之间通信所需要的共享数据结构的设计、数据的传送、进程的互斥与同步等等，都需要程序员去实现



2.6.1 进程通信的类型

高级通信机制可归结为四类：

□ 共享存储器系统

□ 管道通信系统

□ 消息传递系统

- 直接通信方式
- 间接通信方式——信箱通信

□ 客户机-服务器系统

单机(集中式)

适合单机或网络



1. 共享存储器系统（Shared-Memory System）

在共享存储器系统中，相互通信的进程共享某些**数据结构或共享存储区**，进程之间能够通过这些空间进行通信。可分成以下两种类型：

- （1）基于共享数据结构的通信方式。
- （2）基于共享存储区的通信方式。



1. 共享存储器系统（Shared-Memory System）

（1）基于共享数据结构的通信方式

如：生产者-消费者问题中，使用有界缓冲区实现通信。

- ❑ 缺点：操作系统只提供共享存储器，程序员负责公用数据结构的设置及对进程间同步的处理，增加了程序员的负担。属于低效的通信方式（**低级通信**）。



1. 共享存储器系统（Shared-Memory System）

（2）基于共享存储区的通信方式

在存储区中划出一块**共享存储区**，诸进程可通过对共享存储区中数据的读和写来实现通信。——属于**高级通信方式**。

- 先向系统申请共享存储区中的一个分区，并将其附加到自己的地址空间；
- 此后，便可像读写普通存储器那样地读写该公用存储分区。——UNIX/LINUX与之有关的系统调用有4个。

shmget(), shmat(), shmdt(), shmctl()



2. 管道（pipe）通信

首创于UNIX系统

□ 管道：用于连接一个读进程和一个写进程以实现它们之间通信的一个共享文件，又称**pipe**文件。

- 写进程以**字节流**的形式将大量数据送入管道；
- 读进程从管道中接收（读）数据。

由于发送进程和接收进程是通过管道进行通信的，故称为管道通信。



2. 管道（pipe）通信

□ 为协调双方通信，管道机制必须提供以下三方面的协调能力：

- 互斥：当一个进程正在对pipe执行读/写操作时，其它进程必须等待；
- 同步：当写进程把一定数据写入pipe，便去睡眠等待，直至读进程取走数据后，再把它唤醒；当读进程读一空pipe时，也应睡眠等待，直至写进程将数据写入pipe后，再把它唤醒。
- 确定对方是否存在：只有确定对方已存在时，才能进行通信。



3. 消息传递系统(Message passing system)

- 消息传递机制：用得最广泛的一种高级通信机制。包括单机系统、多机系统、计算机网络。
 - 进程间的数据交换,是以**格式化的消息(message)**为单位的。
 - 计算机网络中，message又称为**报文**。
 - 利用系统提供的一组通信命令（**原语**）进行通信。
 - OS隐蔽了通信实现细节，大大简化了通信程序编制的复杂性，因而得到广泛应用。
- 根据实现方式不同，分为：
 - 直接通信方式：
 - 间接通信方式： 共享中间实体（邮箱）



4. 客户机-服务器系统

目前主流的通信实现机制，实现方法分为三类：

- 套接字
- 远程过程调用
- 远程方法调用

□ 套接字（ **socket** ）

一个套接字就是一个通信标识类型的数据结构。分两类：

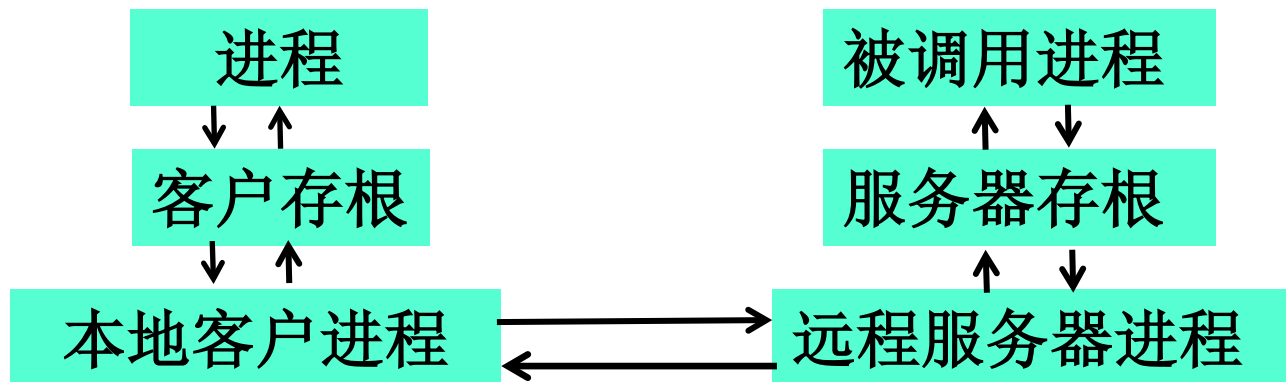
- 基于文件型：同一台主机环境，一个套接字关联一个特殊文件，类似管道。
- 基于网络型：不同主机的网络环境。被分配一对套接字，一个属于接收进程（或服务器），一个属于发送进程（或客户端）。



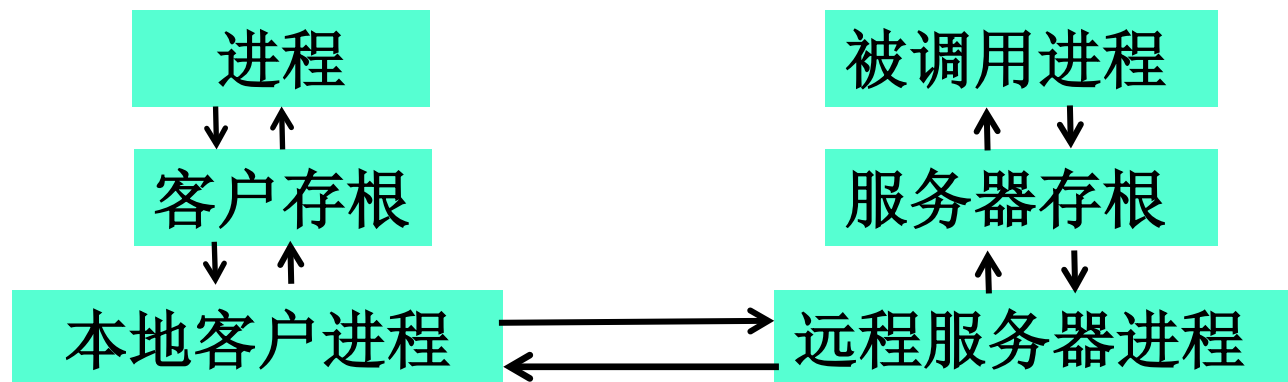
4. 客户机-服务器系统

□ 远程过程调用（RPC）

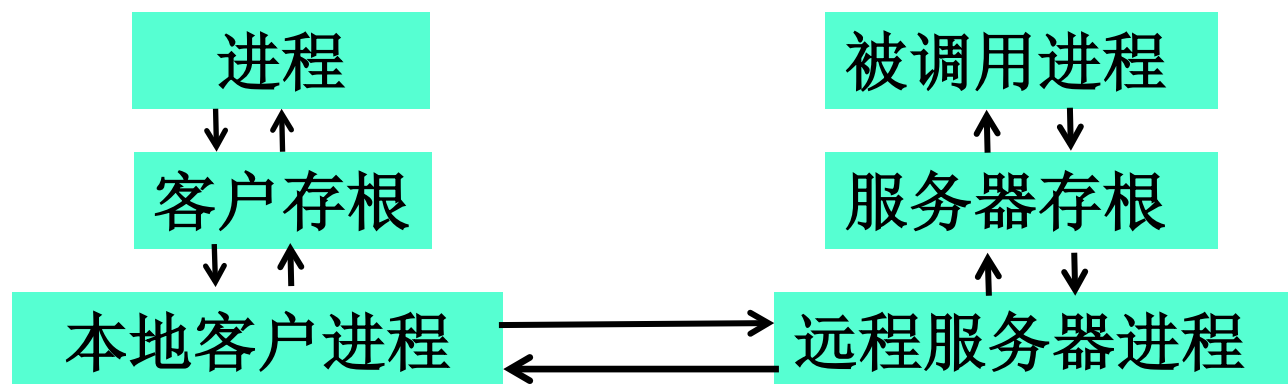
是一个通信协议，用于通过[网络连接](#)的系统。允许一台主机系统上进程调用另外一台主机系统上的进程，程序员表现为常规的过程调用，无需额外地为此编程。如果涉及的软件采用面向对象编程，远程过程调用亦可以成为远程方法调用。



- (1) 本地过程调用者以一般方式调用远程过程在本地关联的客户存根，传递相应的参数，然后将控制权转移给客户存根；
- (2) 客户存根执行，完成包括过程名和调用参数等信息的消息建立，将控制权转移给本地客户进程；
- (3) 本地客户进程完成与服务器的消息传递，将消息发送到远程服务器进程；
- (4) 远程服务器进程接收消息后转入执行，并根据其中的远程过程名找到对应的服务器存根，将消息转给该存根；
- (5) 该服务器存根接到消息后，由阻塞状态转入执行状态，拆开消息从中取出过程调用的参数，然后以一般方式调用服务器上关联的过程；



- (6) 在服务器端的远程过程运行完毕后，将结果返回给与之关联的服务器存根；
- (7) 该服务器存根获得控制权运行，将结果打包为消息，并将控制权转移给远程服务器进程；
- (8) 远程服务器进程将消息发送回客户端；
- (9) 本地客户进程接收到消息后，根据其中的过程名将消息存入关联的客户存根，再将控制权转移给客户存根；
- (10) 客户存根从消息中取出结果，返回给本地调用者进程，并完成控制权的转移。





2.6.2 消息传递通信的实现方式

1. 直接通信方式

发送进程利用OS所提供的命令，直接把消息发送给目标进程。

(1) 对称寻址方式：要求发送和接收进程都以显式方式提供对方标识符。两条通信原语为：

- **send(Receiver, message);**

发送一个消息message给接收进程Receiver

- **receive(Sender, message);**

接收Sender发来的消息message

不足：一旦改变进程的名称，可能需要检查所有其他进程的定义，有关该进程旧名称的所有引用都必须查找到，以便将其修改为新名称。不利于实现进程定义的模块化。



2.6.2 消息传递通信的实现方式

1. 直接通信方式

发送进程利用**OS**所提供的命令，直接把消息发送给目标进程。

(2) 非对称寻址方式：某些情况下，接收进程可能需要与多个发送进程通信时，无法事先指定发送进程。此时，在**接收原语**种只填源进程参数，即完成通信后的返回值。发送和接收原语可表示为：

- **send(P, message);** //发送一个消息message给进程P
- **receive(id, message);** //id是返回值（标识符）



2.6.2 消息传递通信的实现方式

2. 间接通信方式（信箱通信）

□ 进程之间的通信需要通过作为共享数据结构的实体——**信箱**。

- 信箱暂存发送进程发送给目标进程的消息；
- 接收进程从信箱中取出对方发给自己的消息；
- 每个信箱都有一个**唯一的标识符**；
- 消息在信箱中可以安全地保存，只允许核准的目标用户随时读取。

既可实现实时通信，也可实现非实时通信。



2.6.2 消息传递通信的实现方式

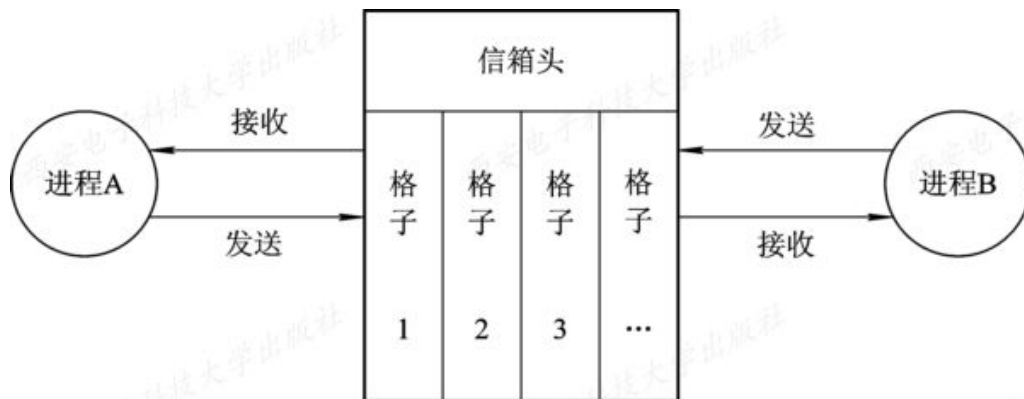
2. 间接通信方式

(1) 信箱的结构

消息传递方式可分为单向邮箱和双向邮箱

信箱定义为一种数据结构。在逻辑上，可以将其分为两个部分：

- 信箱头：存放有关信箱的描述信息，如信箱标识符、信箱的拥有者、信箱口令等。
- 信箱体：由若干个可以存放消息的信箱格组成，信箱格的数目以及每格的大小是在创建信箱时确定的。





2.6.2 消息传递通信的实现方式

2. 间接通信方式

(2) 信箱通信原语

- 信箱的创建和撤消：信箱可由OS创建，也可由用户用OS命令创建。

- 消息的发送和接收：

- **send(mailbox, message);**
- **receive(mailbox, message);**

将一个消息message发送到指定的信箱mailbox

从指定信箱mailbox接收消息message



2.6.2 消息传递通信的实现方式

2. 间接通信方式

(3) 信箱的类型

邮箱可由操作系统创建，也可由用户进程创建，创建者是邮箱的拥有者。据此，可把邮箱分为以下三类：

- 私用邮箱
- 公用邮箱
- 共享邮箱

用户进程可为自己建立一个新邮箱，并作为该进程的一部分。邮箱的拥有者有权从邮箱中读取消息，其他用户只能将自己构成的消息发送到该邮箱中。

私有邮箱可以采用单向通信链路的邮箱来实现

拥有该邮箱的进程结束时，邮箱也随之消失



2.6.2 消息传递通信的实现方式

2. 间接通信方式

(3) 信箱的类型

邮箱可由操作系统创建，也可由用户进程创建，创建者是邮箱的拥有者。据此，可把邮箱分为以下三类：

- 私用邮箱
- 公用邮箱
- 共享邮箱

由操作系统创建，并提供给系统中的所有核准进程使用。

核准进程既可以把消息发送到邮箱中，也可以从邮箱中读取发送给自己的消息。

公用邮箱应采用双向通信链路的邮箱来实现

公用邮箱在系统运行期间始终存在



2.6.2 消息传递通信的实现方式

2. 间接通信方式

(3) 信箱的类型

邮箱可由操作系统创建，也可由用户进程创建，创建者是邮箱的拥有者。据此，可把邮箱分为以下三类：

- 私用邮箱
- 公用邮箱
- 共享邮箱

由某进程创建，在创建时或创建后指明它是可共享的，同时须指出共享进程（用户）的名字。

邮箱的拥有者和共享者都有权从邮箱中取走发给自己的消息



2.6.2 消息传递通信的实现方式

2. 间接通信方式

(3) 信箱的类型

利用邮件通信时，发送进程和接收进程之间存在以下四种关系：

- 一对一关系
发送进程和接收进程建立一条两者专用的通信链路，使两者之间的交互不受其他进程的干扰。
- 多对一关系
允许提供服务的进程与多个用户进程之间进行交互，也称为客户/服务器交互
- 一对多关系
允许一个发送进程与多个接收进程交互，使发送进程可用广播方式向接收者（多个）发送消息。
- 多对多关系
允许建立一个公共邮箱，让多个进程都能像邮箱中投递消息，也可从邮箱中取走属于自己的消息



2.6.3 直接消息传递系统实例

消息缓冲队列通信机制首先由美国的Hansan提出，并在RC 4000系统上实现，后来被广泛应用于本地进程之间的通信中。在这种通信机制中，发送进程利用Send原语将消息直接发送给接收进程；接收进程则利用Receive原语接收消息。



2.6.4 消息缓冲队列通信机制

1. 消息缓冲队列通信机制中的数据结构

(1) 消息缓冲区 其结构可描述如下：

```
struct message_buffer {  
    long sender;           //发送者进程标识符  
    int size;              //消息长度  
    char text[N];          //消息正文  
    struct message_buffer * next //指向下一个消息缓冲区的指针  
};
```




2.6.4 消息缓冲队列通信机制

1. 消息缓冲队列通信机制中的数据结构

(2) PCB中有关通信的数据项

采用消息缓冲队列通信机制时，除了需要为进程设置消息缓冲队列外，还应在进程的PCB中增加消息队列队首指针，以及用于实现同步的互斥信号量mutex和资源信号量sm。

```
typedef struct processcontrol_block {  
    struct message_buff *mq; //消息队列首指针  
    semaphore mutex; //消息队列互斥信号量  
    semaphore sm;    //消息队列资源信号量  
}PCB;
```



2.6.4 消息缓冲队列通信机制

2. 发送原语

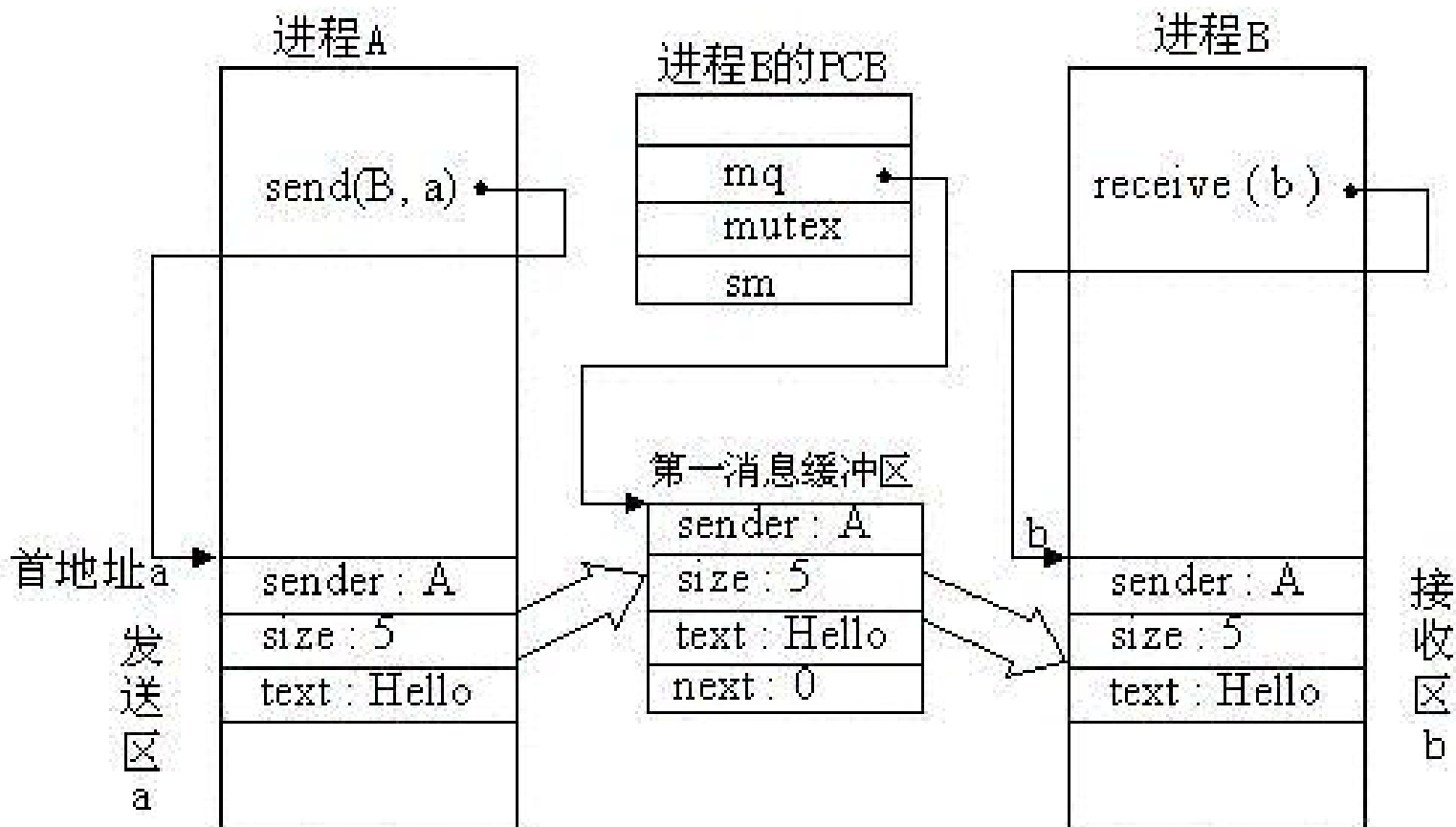
发送进程先在自己的内存空间设置一**发送区a**，填入待发送的消息正文、发送进程标识符、消息长度等信息；然后调用发送原语，把消息发送给目标（接收）进程。

- 发送原语首先根据发送区**a**中所设置的**消息长度a.size**来**申请一缓冲区i**；
- 把发送区**a**中的信息**复制**到缓冲区**i**中；
- 获取接收进程的**内部标识符j(PID)**，然后通过**insert**操作将**i**挂在**j.mq**上，由于该队列属于临界资源，故在执行**insert**操作的前后都要执行**P**和**V**操作



2.6.4 消息缓冲队列通信机制

消息缓冲队列通信原理如图2-17所示





2.6.4 消息缓冲队列通信机制

2. 发送原语

```
send(receiver,a)
{getbuf(a.size,i);           //根据a.size申请缓冲区i
 i.sender = a.sender;
 i.size = a.size;
 i.text = a.text;
 i.next = 0;
}

getid(PCB set,receiver.j); //获得接收进程内部标识符
wait(j.mutex);
insert(j.mq,i);           //将消息缓冲区插入到消息队列中
signal(j.mutex);
signal(j.sm);             //资源数目增1(有可能唤醒接收进程)
}
```

将a中信息复制到消息缓冲区i中



2.6.4 消息缓冲队列通信机制

3. 接收原语

从自己的消息缓冲队列mq 中，摘下第一个消息缓冲区i，并将其中的数据复制到以b为首址的指定消息接收区内。

receive(b)

{j = internal name; //j为接收进程内部标识符

wait(j.sm);

wait(j.mutex);

remove(j.mq,i); //将消息队列中第一个消息i移出

signal(j.mutex);

b.sender = i.sender

b.size = i.size

b.text = i.text

}

将消息缓冲区i中消息
复制到接收区b



2.7 线程的基本概念

□ 2.7.1 线程的引入

□ 2.7.2 线程与进程的比较

□ 2.7.3 线程的状态和线程控制块



2.7 线程的基本概念

- ❑ 20世纪60年代人们提出了进程概念后，在OS中一直都是以**进程**为能拥有资源和独立运行的基本单位的。
- ❑ 直到**20世纪80年代中期**，人们又提出了比进程更小的**能独立运行的基本单位——线程**（Threads）。试图用它来提高系统内程序并发执行的程度，从而进一步提高系统的吞吐量。
- ❑ 进入**20世纪90年代后**，多处理机系统得到迅速发展，线程能比进程更好地提高程序的并发执行程度，充分地发挥多处理机的优越性，因而近几年所推出的多处理机OS，都引入了线程，以改善OS的性能。

OS引入线程，目的是减少程序在并发执行时所付出的时空开销。

2.7.1 线程的引入

进程“太重”，系统在进程上所花的时空开销大致表现在：

- 1) 创建进程 系统在创建进程时必须为它分配除**CPU**以外的所有资源，如内存空间、I/O设备，以及建立相应的**PCB**。
- 2) 撤消进程 系统撤消进程时，必须先回收其所占的资源，然后在撤消**PCB**。
- 3) 进程切换 进程切换时，由于要保留当前进程的**CPU**环境和设置新进程的**CPU**环境，因而需花费不少处理机时间。

总之，由于进程是一个资源拥有者，因而在创建、撤消、切换中，系统必须为之付出较大的时空开销，故系统中的进程数目不宜过多，进程切换的频率也不宜过高，这限制了并发程度的进一步提高。



2.7.1 线程的引入

进程既是资源分配的基本单位，又是调度和分派的基本单位。若能将进程的上述**两个属性分开**，由操作系统分开处理，即：

★对于作为调度和分派的基本单位，不同时作为拥有资源的基本单位，以做到“轻装上阵”；

★而对于拥有资源的基本单位，又不对之进行频繁的切换。

在这种思想指导下，形成了线程概念。线程只作为调度和分派的基本单位，而不作为资源分配的基本单位。一个进程通常包括多个线程。



2.7.2 线程与进程的比较

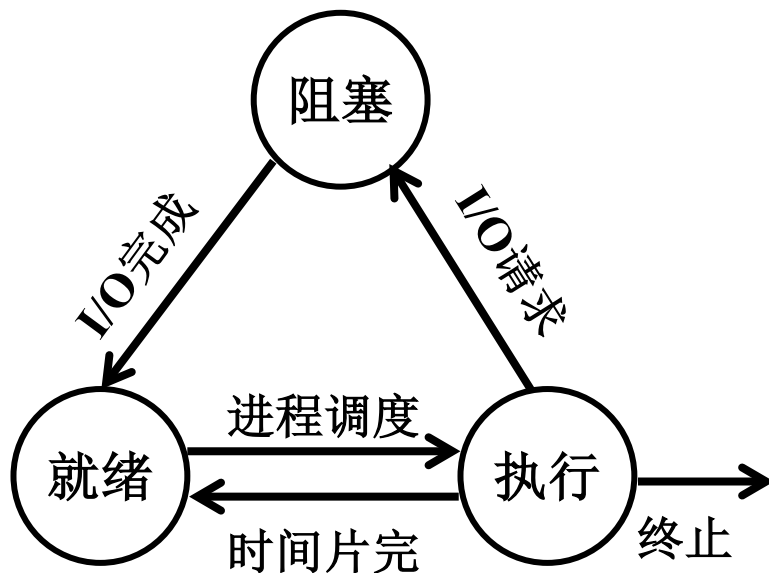
传统进程称为重型进程，线程称为轻型进程。

- 调度基本单位 $\left\{ \begin{array}{l} \text{进程: 传统OS} \\ \text{线程: 现代OS} \end{array} \right.$
- 并发性 $\left\{ \begin{array}{l} \text{进程: 可并发} \\ \text{线程: 同一进程的多个线程甚至全部线程可并发执行;} \\ \text{不同进程的线程也可并发执行} \end{array} \right.$
- 拥有资源 $\left\{ \begin{array}{l} \text{进程: 拥有资源, 是系统资源分配的基本单位} \\ \text{线程: 仅有一点必不可少、能保证独立运行的资源} \end{array} \right.$
- 独立性 同一进程中的不同线程共享进程内存空间和资源
- 系统开销 线程创建、撤销、切换以及通信的开销明显小于进程
- 支持多处理机系统 一个进程的多个线程可分配到多个处理机上



2.7.3 线程的状态和线程控制块

1. 线程运行的三个状态





2.7.3 线程的状态和线程控制块

2. 线程控制块（TCB）

系统为每个线程配置一个线程控制块，包括：

- 线程标识符
- 一组寄存器
- 线程运行状态
- 优先级
- 线程专用存储器
- 信号屏蔽
- 堆栈

复习



1. 某系统的进程状态如图所示：

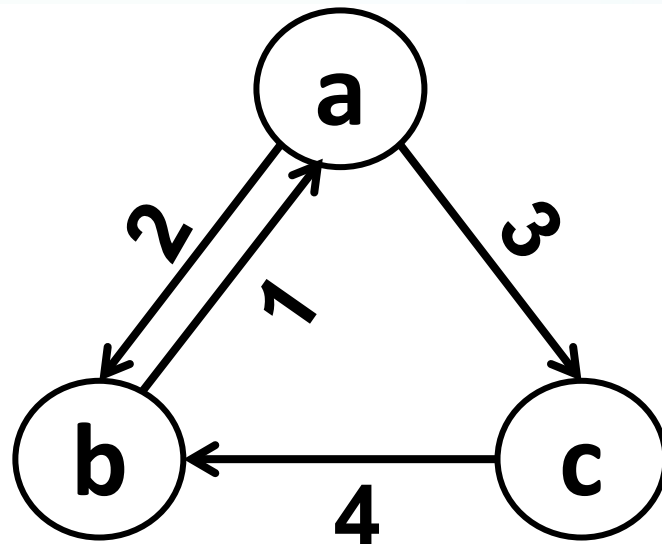
a是_____状态，b是_____状态，c是_____状态。

1表示_____，2表示_____，3表示发生了等待事件，

4表示等待事件结束。（填写“选中”或“落选”）

下列情况中，当发生前者的状态变化时，会导致发生后者的状态变化的。

A. $2 \rightarrow 1$ B. $4 \rightarrow 2$



2. 在操作系统中，进程是一个具有一定独立功能的程序在某个数据集上的一次（ ）

- ☐ A 等待活动
- ☐ B 单独操作
- ☒ C 运行活动
- ☐ D 关联操作

提交



3. () 是解决进程间同步和互斥的一对低级通信原语。

- ☐ A lock和unlock
- ☒ B P和V
- ☐ C W和S
- ☐ D Send和Receive

提交



4. 下面对进程的描述中，错误的是（ ）。

- ☐ A 进程是动态的概念
- ☐ B 进程执行需要处理机
- ☐ C 进程是有生命期的
- ☒ D 进程是指令的集合

提交



5. 信箱通信是一种 () 通信方式。

- ☐ A 直接通信
- ☒ B 间接通信
- ☐ C 低级通信
- ☐ D 信号量

提交



6. 操作系统通过 () 对进程进行管理。

- ☐ A JCB
- ☒ B PCB
- ☐ C DCT
- ☐ D CHCT

提交



7. 一个进程被唤醒意味着（ ）。

- ☐ A 该进程重新占有了CPU
- ☐ B 它的优先权变为最大
- ☐ C 其PCB移至等待队列队首
- ☒ D 进程变为就绪状态

提交



8. 下面所述步骤中，（ ）不是创建进程所必需的。

- ☒ A 由调度程序为进程分配CPU
- ☐ B 建立一个进程控制块
- ☐ C 为进程分配内存
- ☐ D 将进程控制块链入就绪队列

提交



9. 多道程序环境下，操作系统分配资源以（ ）为基本单位。

- ☐ A 程序
- ☐ B 指令
- ☒ C 进程
- ☐ D 作业

提交





河南大學
HENAN UNIVERSITY

明德，新民
止于至善



The End ...

